



## Undergraduate FYP Proposal Form<sup>1</sup>

### Section I: Student's Information:

|            |                                |           |               |
|------------|--------------------------------|-----------|---------------|
| Full Name  | Al-Kubati Aiham Ammar Abdullah |           |               |
| Student ID | SWE2302173                     | Mobile    | +601169958529 |
| Email ID   | swe2302173@xmu.edu.my          | Programme | SWE           |

### Section II : Industry Information: (Complete this section if your project is going to be conducted in partnership with any company)

|                |  |       |  |
|----------------|--|-------|--|
| Company Name   |  |       |  |
| Contact Person |  |       |  |
| Designation    |  | Phone |  |
| Email ID       |  |       |  |

### Section III : Declaration by the Student

By signing this form, I confirm that I have read and will adhere to the **Final Year Project Proposal Guidelines** of Xiamen University Malaysia as applicable to this application.

Signature of Student: \_\_\_\_\_

Date: 12 November 2025

### Section VI : Supervisor Approval

|                    |                             |      |                 |
|--------------------|-----------------------------|------|-----------------|
| Name of Supervisor | Geetha Kanaparan            |      |                 |
| Office Location    | A1#405                      |      |                 |
| Email Address      | geetha.kanaparan@xmu.edu.my |      |                 |
| Office Extension   |                             |      |                 |
| Signature          |                             | Date | 9 December 2025 |

<sup>1</sup> The length of the proposal should be between 2000-3000 (Excluding proposed title & references)

## SECTION V : Proposal Information

### **Proposed Title** *(Capitalise each word)* **(Not more than 20 words)**

An Interactive Graph-Based Visualization Approach to Improving Source Code Comprehension and Developer Onboarding

### **Introduction**

*(Introduce the context & research gaps/problem)* **(300-400 words)**

Onboarding contributors to ongoing complex software projects remains as a significant challenge, which leads to higher cognitive effort and extended periods of time for new developers to a project to solve programming tasks effectively. This challenge stems from the expanding complexity and varying architectures of modern software systems. Insufficient comprehension during onboarding is linked to increased bugs and slower development (Yates et al., 2020).

In recent years, there has been a growing interest in exploring methods for streamlining software comprehension. Previous investigations include the development and evaluation of graph-based software visualization tools to aid in source code comprehension. Despite substantial progress in the field, literature identifies three different limitations that restrict the effectiveness and future improvement of current software visualization tools in streamlining software source code comprehension: (1) Programmers tend to switch frequently between the program model and the situation model when studying code (Von Mayrhauser et al., 1997). However, many studies neglect this semantic gap in software visualization tools. There is a scarcity of tools that associate high-level domain concepts with low-level code logic, forcing contributors to spend cognitive effort in mentally reconstructing mappings between the two levels. (2) There is a notable lack of empirical evaluation of tools involving realistic maintenance tasks on industrial software systems. According to a systematic literature review by (Merino et al., 2018), only 12 out of 181 reviewed papers on software visualization tools include real-world industrial software systems in their evaluation. This limits the generalizability of findings to the industrial environment and substantiating the applicability of the tool. (3) Current software visualization tools' evaluations predominantly focus on performance metrics such as completion time. There is a lack of comprehensive assessments and considerations to factors such as user experience and cognitive load (Goncales et al., 2019), which can obscure potential opportunities for future improvement.

Past studies indicate various possible solutions to the problem, with an abundance of literature focusing on graph-based source code visualization tools to streamline the onboarding experience for software project contributors by reducing the cognitive effort spent comprehending unfamiliar software systems, thereby reducing the time taken for onboarding contributors to perform successful changes on the software source code. This project is to focus on this aspect of the project.

## **Aims and Objectives**

*(What & how) (100-150 words)*

The general main aim of this project is to streamline software project contributor onboarding by reducing the taken time to comprehend software source code and taking actions towards unfamiliar software code in making successful changes such as debugging and adding new features. The main objectives of the project are as listed below:

- To design and outline a standalone application that is targeted towards streamlining developer onboarding based on the identified techniques, trends and principles in the study.
- To develop a multi-level visualization model that allows the developer to navigate from high-level architectural views into low-level, detailed overviews of functions, classes, dependencies, etc.
- To conduct a controlled user study to measure the tool's effectiveness against realistic scenarios, evaluating the tool by measuring performance metrics, cognitive load and user experience metrics.

## **Literature Review**

*(Related works) (1000-1300 words)*

### **Challenges in Software Source Code Comprehension**

Onboarding is faced constantly in the industry during the software development lifecycle. It is an essential process that involves getting software project contributors to comprehend software systems to perform successful tasks on the project. A major part of software contributor onboarding involves studying the software system's logic and structure, leading to onboarding being a costly and time-consuming complication despite its high prevalence in the industry. Various studies have been made on traditional industry onboarding techniques. Based on empirical data by Buchan et al. (2019), onboarding new contributors to a software project is perceived to range between 2 to 6 months using various traditional techniques such as mentoring and documentation. This empirical study has also shown that onboarding techniques generally revolve around 11 main goals, the most notable of them being G6 "Understand the project structure and aims and the implications" and G11 "Understand the project domain knowledge and terminology". These goals are to be considered as the main objectives of a software source code visualization tool for the project.

The prolongation of the onboarding process is a result of the high cognitive load an onboarding contributor experiences with comprehending and navigating unfamiliar source code. A systematic mapping review by Goncales et al. (2019) implies that cognitive load assessment can be, and is often used by many studies to gauge factors such as the productivity of contributors during programming tasks. Similarly, another systematic review by Merino et al. (2018) argues that software visualization tools' evaluations should not only focus on performance metrics (e.g., task completion time), but other factors such as usability and the engagement level of the user. Therefore, such qualities need to be heavily considered in the design of a tool to streamline contributor onboarding.

### **Current Practices in Software Visualization Tools**

A number of studies has examined various tool designs to facilitate software comprehension by visualization for their effectiveness. A considerable body of work generally employs a foundational concept of ontology-based graph models to represent the logical structure of the software project. This method includes visualizing software source code as node-link diagrams (Nagel et al., 2021), in which different types of nodes represent entities such as classes, functions, variables, etc., while links represent dependencies/relationships between the different entities. For example, Richard Wettel et al. (2011) adopts this concept into a 3-dimensional approach by developing a tool to visualize software source code based on a city metaphor, representing classes as buildings, packages as districts, etc. A key strength of such approach is its implementation of real-world spatial analogies, which facilitates the perception of the user on the scale of different entities and their attributes in the codebase. However, the experimentation of the tool did not include examining the reliability of the tool for visualizing large-scale industry software, which is a common complication that needs to be looked into to assess the usability of the tool in the real world. A different study by Stepanek et al. (2024) explores an alternative approach to tackle the scalability issue, in which the node-link graph visualization follows an aggregation mechanism in which nodes can be expanded, collapsed and filtered on demand in order to accommodate for large-scale software project while ensuring readability. The tool design also includes the extensive implementation of icons, colours, and other visual indicators to represent different information relating to the program entities, such as the colour of the node representing the type of the entity (function, class, etc.). Despite the use of an intuitive approach to overcome large-scale source code, the study claims that the tool continues to struggle with scalability complications as well as performance. The tool is also designed to solely support C# projects.

Projects such as Graph Buddy (Borowski et al., 2024) acknowledge the high-level view principle and presents the Semantic Code Graph (SCG), a software visualization model targeted to simplify software comprehension tasks. SCG uses a rule-based framework that identifies specific relationships, overridden functions, etc. of the software project structure that contribute to the high-level overview of the system, presenting a more semantic graph, which is particularly valuable for onboarding and software source code comprehension tasks. However, SCG fails to represent architectural patterns in the software, such as the dependencies, function calls, and other logical artifacts, making it only effective in high-level views of the system. However, relying solely on a single level of detail in software visualization may have its drawbacks. A study by Howard (2000) specifies a quality found to be desirable in software visualization: Data to be visualized must be presented at several levels of detail, starting from a broad overview of the system, and gradually transitioning into the fine structure. The study also mentions the need to reward close attention of the user at a specific body in the graph with more information. Findings by a study exploring program understanding behaviour (Von Mayrhauser et al., 1997) further supports the evidence by concluding that programmers tend to switch frequently between the domain concepts of the program and the program model when attempting to comprehend software code.

Therefore, this highlights the need for a new visualization tool with the priority of bridging a semantic domain view and a structural view, which will work more effectively in software source code comprehension.

### **Simplifying Navigation and User Interface Considerations**

Interaction design can heavily influence the cognitive effort on contributors when using the visualization tool to understand software code. Hence, it is vital to ensure that accurate consideration is given to the experience of the developer with the tool, and not only their time taken to successfully complete tasks in the software project. For example, Wandercode (Henley et al., 2024), a code recommender design with an interactive graph visualization tackles the issue of cognitive overload by displaying the user interface structure overlapping with the developer's IDE, ensuring high interface coupling between visualization tool and the coding space for the developer which reduces the cognitive effort needed to use the tool.

As for the graph-based visualization, it is clear that multiple designs of discussed literature agree it is vital to include intuitive functions such as filtering, zooming, and navigating around the graph as implemented in their resultant tools (Borowski et al., 2024; Richard Wettel et al., 2011; Stepanek et al., 2024). These functions can facilitate navigation and assist the developer in adjusting their view to the relevant sections of the graph based on their needs, hence would need to be included. As earlier discussed, it is also vital to integrate both domain concepts and structural elements of the system together, as developers often switch between them frequently when comprehending a system (Von Mayrhauser et al., 1997), despite a few tool designs including this concept. Adopting the principle indicated by Stepanek et al. (2024), it is also essential to ensure that different visual cues are used to indicate information within the graph.

### **Research Gaps & Conclusion**

Based on the literature reviewed, many of the developed tools have shown success in their evaluations. Furthermore, a critical gap exists in bridging between the semantic model and the structural model of the software source code. Tools such as Helveg (Stepanek et al., 2024) excel at structural visualization and Graph Buddy (Borowski et al., 2024) at semantic visualization. This separation fails to address the observed behaviour by (Von Mayrhauser et al., 1997) on developers frequently switching between the domain model and the structural logic. Hence, there is a clear need for a software visualization tool that bridges between the software project's domain and logic, requiring less cognitive load on onboarding contributors and ultimately streamlining their onboarding phase.

The evaluation methodology and empirical research done on many studies yet presents another limitation. As highlighted by the systematic literature review (Merino et al., 2018), there is a lack of rigorous evaluation of the developed tools with industry software. Moreover, many of the evaluation procedures employed neglected qualitative aspects such as user experience and cognitive load. Without measuring the cognitive data of the contributor's experience, it can hinder the integrity of the determination that the tool can speed up onboarding completion time, since there is a possibility that the use of the tool increases mental strain, which is considered as counter-productive for effective contributor onboarding.

## Research Methodology

*(Data collection / system design, tools & technologies / datasets, & Evaluation methods  
(Ex. testing, user study, simulation) (300-500 words)*

This project will follow the design science methodology framework as defined by Peffers et al. (2007). The first phase includes further delving into the research problem and further justifying the value of a solution, followed by gathering a comprehensive set of objectives on what the prototype should be able to do. The third phase includes the technical execution, which is as follows:

- **Core Architecture:** The program will be implemented as a standalone application (using Electron wrapper) to be a usable prototype for developers of any IDE. As a future improvement, the system can be further implemented as an IDE plugin to address the principles discussed by Henley et al. (2024) and Howard (2000)
- **Front-end Design:** Made using the React framework, with the use of node-link visualization libraries such as cytoscape.js or D3.js.
- **Back-end:** To process software source code files, a programming language parser library will be used (Tree-sitter or scrML), which are able to effectively build syntax trees for multiple languages. The languages to be supported in the prototype design are Python, Java and Javascript.

As for the semantic clustering of data, an AI-powered pipeline can be used, starting with HDBSCAN for automatic cluster detection for domain clusters of the software system (e.g., “Billing Module”), followed by Microsoft’s CodeBERT or GraphCodeBERT for clustering of different files, classes, functions, etc., into their respective semantic feature under the software’s system.

After a working prototype has been developed, the evaluation phase of the tool is to be by tasking selected test subjects on debugging existing open-source software source code that is unfamiliar to the test subject using the prototype tool to comprehend the software source code structure. The test subject selection criteria will preferably target senior students and developers in the industry. The procedure will include realistic tasks such as finding the source of a pre-prepared, existent bug in two different sessions:

1. With the use of only their IDE.
2. With the use of their IDE and the prototype tool to navigate and study the code.

To ensure realism, real projects will be used in the evaluation rather than dummy projects (Merino et al., 2018). The data to be obtained from the evaluation sessions will include the following metrics:

- **Performance:** Task completion time, success rate.
- **Cognitive Load:** NASA-TLX questionnaire (Goncales et al., 2019) after each session.
- **Usability:** System Usability Scale (SUS) at the end of the second session to measure user experience (Howard, 2000; Merino et al., 2018).

- **Qualitative Feedback:** Semi-structured interview to gather insights on strengths and weaknesses of the visualization tool, particularly regarding the aspect of bridging of the domain and code.

After evaluation data has been fully compiled, the final phase of the design science methodology is to communicate the findings obtained from analysing the problems, designing the artifact, and a discussion of the utility of the tool to onboarding software project contributors.

### **Scope & Limitations** *(100-150 words)*

This project targets the research and development of a standalone desktop tool for visualizing software source code for streamlined comprehension. The scope includes the use of existing libraries and AI models to create an interactive navigation system for the user. However:

- the tool will initially focus on supporting Python, Java and Javascript codebases, as supporting more codebases may require more time to implement.
- Exceedingly large enterprise software systems may experience performance constraints when attempting to be visualized by the tool, as it poses higher complexity to the processing of the codebase.
- The AI clustering pipeline accuracy may differ based on factors such as code quality and naming conventions applied to the source code's files, classes, functions, variables, etc.
- The visualization tool focuses on architectural comprehension but does not fully replace code reading, and is rather encouraged to be used alongside an IDE.

### **Potential Project Significance** *(100-200 words)*

This project holds high significance towards improving the practice and the study of software contributor onboarding. By addressing the identified research gaps, especially the divide between the semantic domain representation and code structure, it contributes a valid solution that matches contributor demands when comprehending software structures based on referenced studies (Von Mayrhauser et al., 1997). In practice, the use of this tool can reduce the cognitive load on contributors and allows them to produce successful changes to unfamiliar source code in less taken time. This can ultimately lower the barrier for new contributors, external software development teams, cybersecurity analysts, QA engineers, and any other segregated teams involved in the development, analysis, or maintenance of open-source and industrial software projects, leading to more sustainable software development.

### **Expected Outcomes & Concluding Remarks**

*(Deliverables, expected results & conclusions) (200-300 words)*

The primary deliverable of this project will be a functional, standalone desktop application for graph-based software source code visualization. The application will be accessible as an open-source codebase (through GitHub) and the research is to include a comprehensive evaluation dataset based on the evaluation methodology. The tool is expected to demonstrate several key outcomes:

- A measurable reduction in the time taken for onboarding contributors to complete initial comprehension tasks.
- A significant decrease in their reported cognitive load compared to using standard IDEs alone for debugging of unfamiliar code.
- Effective bridging of high-level domain concepts with low-level code structure through the AI-powered clustering pipeline.

This project will validate the hypothesis that interactive, semantically-aware graph visualizations can streamline the onboarding process for contributors. It will also provide empirical evidence that such tools can be practical instruments to reduce cognitive load for comprehending unfamiliar codebases. The research will conclude by demonstrating that a focus on user-centric metrics like cognitive load and usability with traditional performance metrics is crucial for developing effective tools targeting contributors in software engineering tasks.

Ultimately, this project aims to deliver a robust proof-of-concept that transitions academic research into a potential tool to be used by internal stakeholders to software project. By linking the design of the visualization to streamlined comprehension, it will lay a foundation for future work and research in intelligent developer tools and set a new standard for the rigorous evaluation of related systems within the software engineering community.

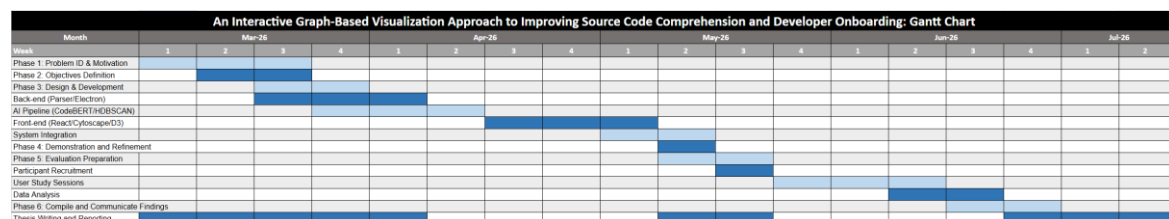
### **Key References**

*(Recent references APA 7<sup>th</sup> Edition) (10-20 references APA style)*

Borowski, K., Balis, B., & Orzechowski, T. (2024). Semantic Code Graph - An Information Model to Facilitate Software Comprehension. *IEEE Access*, 12, 27279–27310. <https://doi.org/10.1109/ACCESS.2024.3351845>



- ## Gantt Chart



---

<sup>2</sup> Please follow APA style of referencing.